Keyboard Control with ROS 2

RC Car Assembly Manual Series No.4

This manual is the fourth one of our RC car assembly manual series. In this manual, we will explain how to install ROS 2 foxy on Ubuntu 20.04 and to construct ROS system to control the RC car. Please read and follow the instructions carefully. There may be changes in parts due to future discontinuation issues.

RC Car Assembly Manual Series

0. Battery Safety Rules Manual
1. RC Car Hardware Assembly Manual
2. VESC Setup Manual
3. Jetson Xavier NX Setup Manual
4. Keyboard Control with ROS2
5. RC Car Calibration Manual

RLLAB

http://rllab.snu.ac.kr

Last modified: 26.10.15

Table of contents

1. Install ROS 2 foxy on Jetson Xavier NX with Ubuntu 20.04
    1.1. Install ROS 2 foxy
    1.2. Install colcon

2. Construct ROS system to control RC car
    2.1. Install vscode
    2.2. Make ROS 2 workspace
    2.3. Download RC car control code and install required packages
    2.4. Connect LiDAR and VESC
    2.5. RC car keyboard control

3. About RC car ROS 2 system code
    3.1. Overview of the system
    3.2. Important nodes

1. Install ROS 2 foxy on Jetson Xavier NX with Ubuntu 20.04

1-1. Install ROS 2 foxy

*The following instruction is described in the **"setup source"** and **"install ros2 package"** part of the official instruction to download ROS 2 foxy on Ubuntu 20.04. (ROS 2 Foxy is compatible with Ubuntu 20.04 (Focal Fossa))*
Instruction link: https://docs.ros.org/en/foxy/Installation/Ubuntu-Install-Debians.html

1-1-1. Setup Sources

First, ensure that the Ubuntu Universe repository is enabled.

```
sudo apt install software-properties-common
```

```
sudo add-apt-repository universe
```

*** *Troubleshooting* ***

If you encounter the message, "Could not get lock /var/lib/dpkg/lock-frontend," try the following commands:

1) Kill the running apt processes

```
sudo killall apt apt-get
```

2) Remove directories

```
sudo rm /var/lib/apt/lists/lock
```

```
sudo rm /var/cache/apt/archives/lock
```

```
sudo rm /var/lib/dpkg/lock*
```

3) Update

```
sudo dpkg --configure -a
```

```
sudo apt update
```

Add the ROS 2 GPG key with apt.

```
sudo apt update && sudo apt install curl -y
```

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
```

Then add the repository to your sources list.

```
ARCH=$(dpkg --print-architecture);
KEY=/usr/share/keyrings/ros-archive-keyring.gpg;
CODENAME=$(. /etc/os-release && echo $UBUNTU_CODENAME);
URL=http://packages.ros.org/ros2/ubuntu;
LINE="deb [arch=$ARCH signed-by=$KEY] $URL $CODENAME main";
echo "$LINE" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null;
```

1-1-2. Install ROS 2 packages

Update your apt repository caches after setting up the repositories.

```
sudo apt update
```

```
sudo apt install ros-foxy-desktop python3-argcomplete
```

1-1-3. Try some examples

In one terminal, source the setup file and then run a C++ talker:

```
source /opt/ros/foxy/setup.bash
```

```
ros2 run demo_nodes_cpp talker
```

In another terminal source the setup file and then run a Python listener:

```
source /opt/ros/foxy/setup.bash
```

```
ros2 run demo_nodes_py listener
```

You should see the “talker” saying that it’s “Publishing” messages and the listener saying “*I heard*” those messages. This verifies both the C++ and Python APIs are working properly.

1-2. Install colcon

To use ROS 2, we need to install colcon. Colcon is a tool used for managing packages related to ROS2. By using colcon, you can set up a ROS 2 workspace, build packages, manage package dependencies, install packages, and handle launch files, among other tasks.

Install colcon with commands:

Caution : Please do not enter “sudo apt upgrade” after update!!!!

```
sudo apt update
```

```
sudo apt install python3-colcon-common-extensions
```

2. Construct ROS system to control RC car

2-1. Install VS code

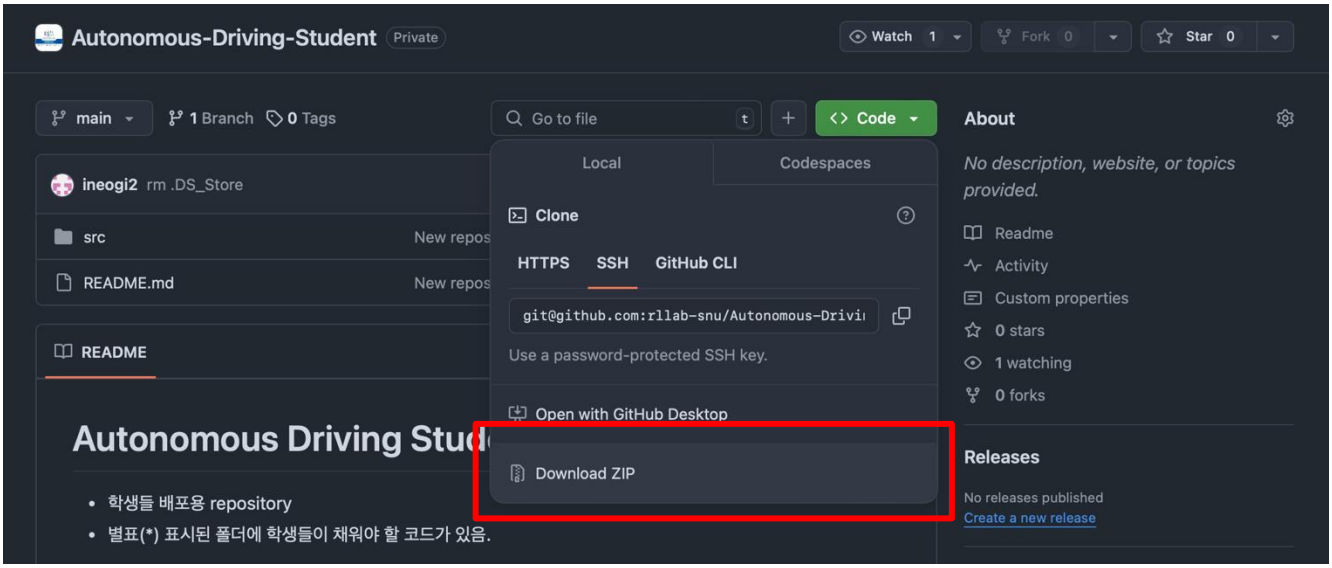
We first recommend you install visual studio code (VS code), a powerful code editor from Microsoft. You cannot install it from the official homepage since the latest version is not supported on arm-based device including Jetson Xavier NX.

Please download the previous version (v1.65.2) that supports arm64 device from the link. (https://update.code.visualstudio.com/1.65.2/linux-deb-arm64/stable)

```
sudo apt install ~/Downloads/code_1.65.2-1646922911_arm64.deb
```

2-2. Download codes & install required packages

First, download our project code from our github.



Change the folder name from “Autonomous-Driving-Student” to “ros2_ws”.

```
git clone https://github.com/jwonkong/Autonomous-Driving-2026.git ros2_ws
```

We introduce rosdep, a tool to manage required ROS 2 packages automatically.
First, install rosdep with command:

```
sudo apt install python3-rosdep2
```

Erase repeated file for proper initialization.

```
sudo rm -rf /etc/ros/rosdep/sources.list.d/20-default.list
```

Initialize rosdep.

```
sudo rosdep init
```

Update cache of rosdep.

```
rosdep update --include-eol-distros
```

Navigate to ros2 workspace.

```
cd ros2_ws
```

Install ROS 2 packages required for the project.

```
rosdep install -i --from-path src --rosdistro foxy
```

*** *Troubleshooting* ***

If you encounter an error while installing ros2 packages, install required packages one-by-one, manually.

```
sudo apt install ros-foxy-ackermann-msgs
```

```
sudo apt install ros-foxy-diagnostic-updater
```

```
sudo apt install ros-foxy-serial-driver
```

```
sudo apt install ros-foxy-urg-node
```

Activate a virtualenv that can contain required python packages for the project.
(You can reuse previous virtualenv we made in Jetson Xavier XN board setup manual which contains pytorch. Make sure that pytorch is installed in a virtualenv if you create a new one.)

```
source ~/venv/torch210/bin/activate
```

Install required python packages.

```
(torch210) cd src/f1tenth
```

```
(torch210) pip install -r requirements.txt
```

```
(torch210) pip install setuptools==58.2.0 empy==3.3.4 lark
```

```
(torch210) pip install catkin_pkg
```

Open ~/.bashrc file with gedit.

```
gedit ~/.bashrc
```

And add the following lines to the end of the file to let your system know where the required python packages exist. Make sure that user_name and virtualenv_name should be certificated with your own ones.

```
export PYTHONPATH=/home/{user_name}/venv/{virtualenv_name}/lib/python3.8/site-packages:$PYTHONPATH
source /opt/ros/foxy/setup.bash
source ~/ros2_ws/install/setup.bash
```

To prevent your ROS 2 communication from mixing with other teams, add the following line as follows:

```
export ROS_DOMAIN_ID=<Your team name>
```

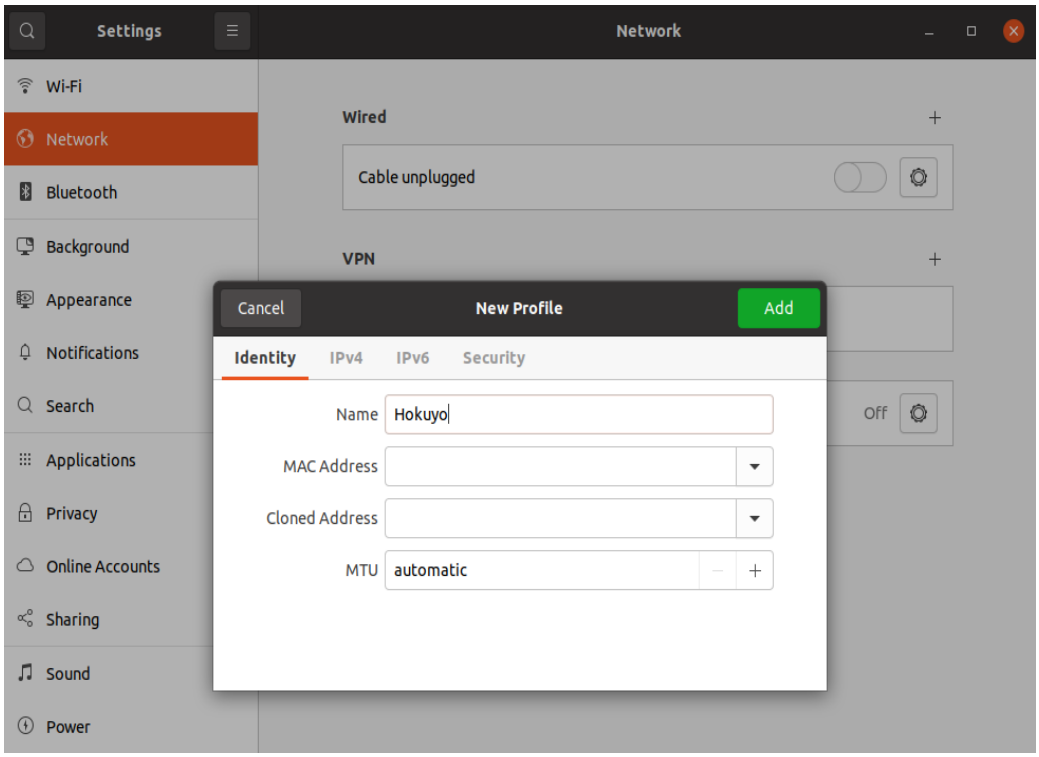
Example : export ROS_DOMAIN_ID=8.
Save and exit.

Apply the changes in the ~/.bashrc file in current terminal.

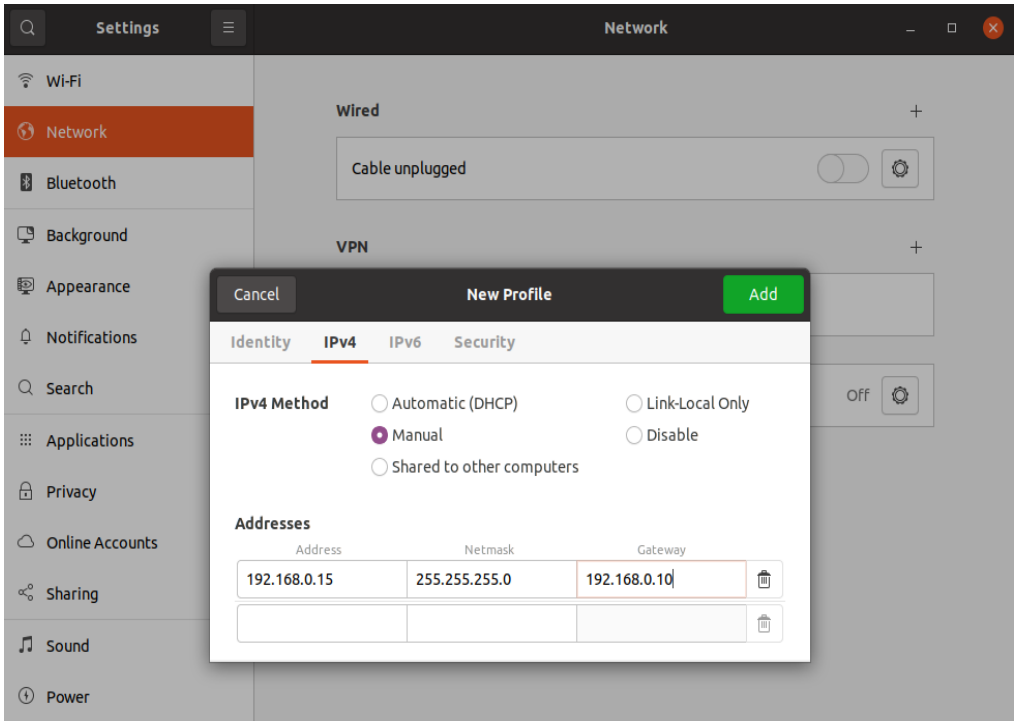
```
source ~/.bashrc
```

2-4. Connect LiDAR and VESC

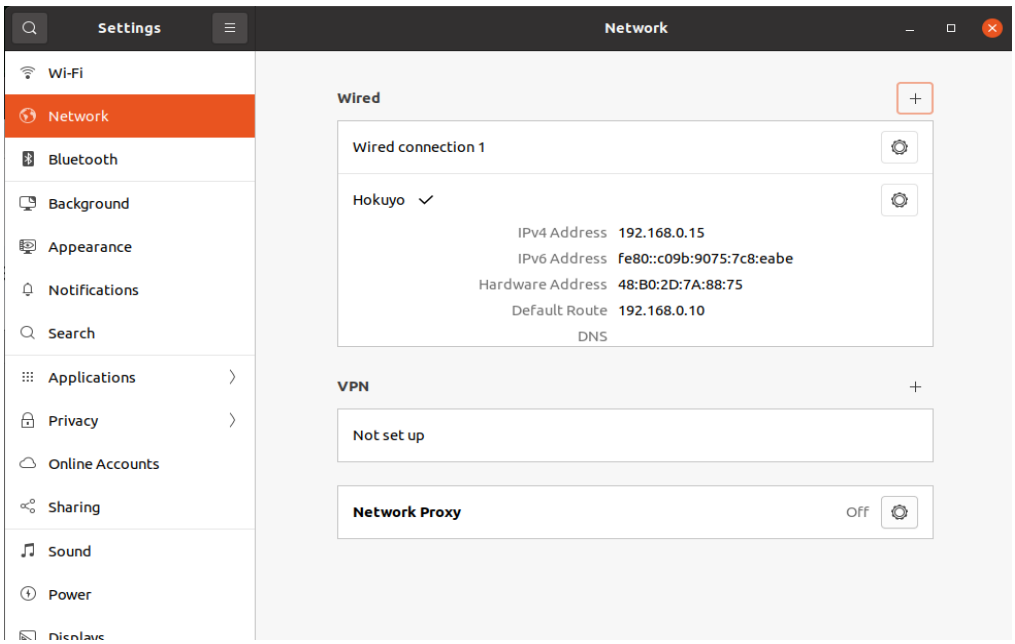
Initially, the LiDAR and VESC are not recognized by the Jetson board. Additional steps are required to ensure that these two components are detected by the Jetson.
Since our LiDAR, Hokuyo UST-10LX, is connected to Jetson board with ethernet connector, go to the "*Network*" tab in the settings, click the "+" button in the "*Wired"* section, and set the Name as Hokuyo.



In the IPv4 tab, select Manual as the method, then enter 192.168.0.15 in the Address field, 255.255.255.0 in the Netmask field, and 192.168.0.10 in the Gateway field.



Finally, click the Add button to save the configuration.



To connect VESC, open /etc/udev/rules.d/99-vesc.rules file

```
sudo gedit /etc/udev/rules.d/99-vesc.rules
```

and type

```
KERNEL=="ttyACM[0-9]*", ACTION=="add", ATTRS{idVendor}=="0483",
ATTRS{idProduct}=="5740", MODE="0666", GROUP="dialout",
SYMLINK+="sensors/vesc"
```

save and exit. And you have to

Caution : The rule in 99-vesc.rules must be written on a single line. Do not add line breaks. Be careful when copying it from the PDF, because it may be pasted as multiple lines. After pasting, check that the rule is on one line.

Reload the system rules.

```
sudo udevadm control --reload-rules
```

```
sudo udevadm trigger
```

```
source ~/.bashrc
```

And you have to unplug the VESC USB cable and plug it back in.

2-5. RC car keyboard control

First, build the whole system files.

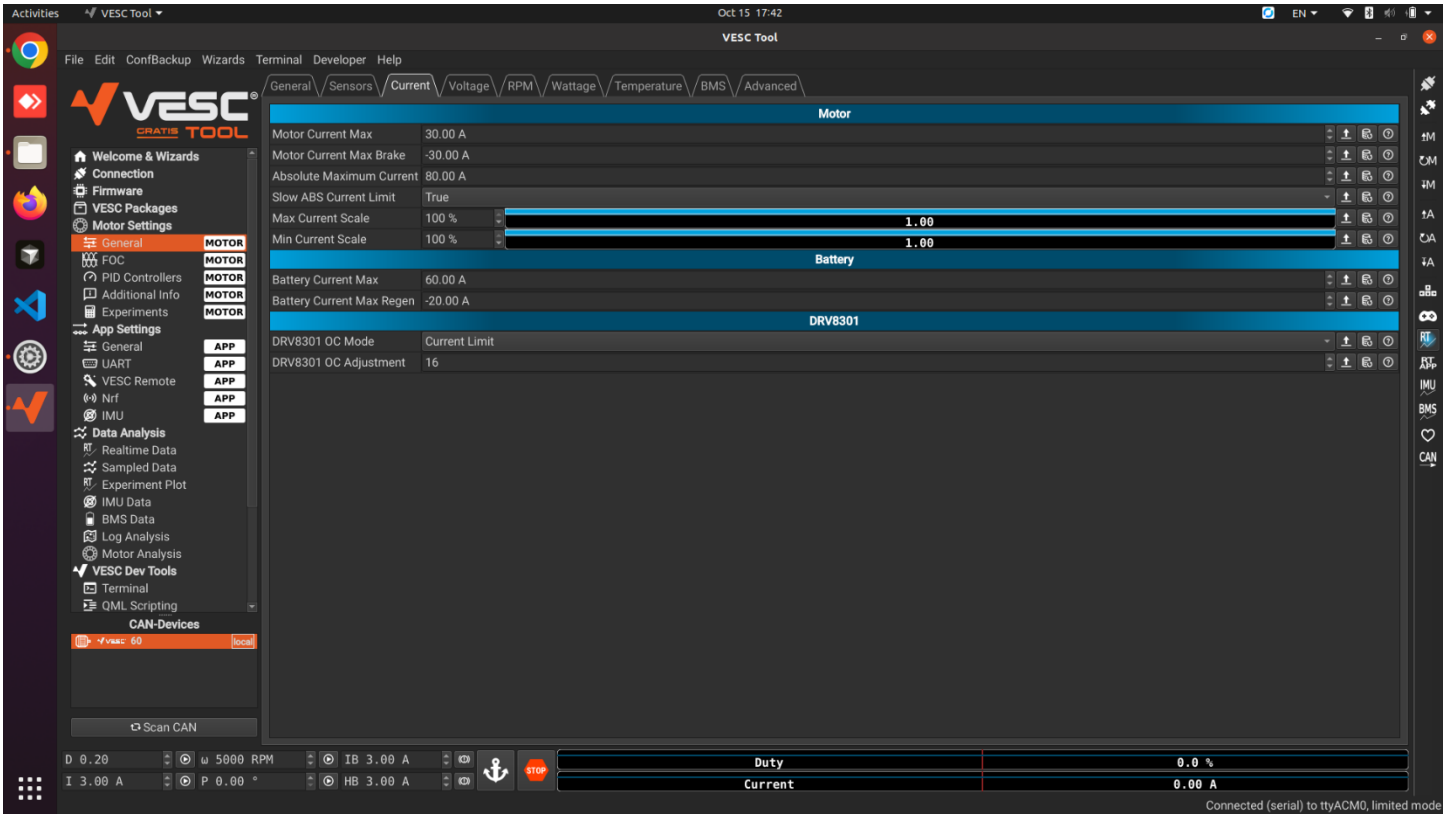
```
colcon build --symlink-install
```

Here the symlink-install option makes it possible to modify your python code without additional ‘colcon build’ process. You only need to re-build the system with ‘colcon build’ when there is a change in the C++ code.

Now you are ready to control your RC car with a keyboard. First, launch the whole RC car ROS 2 system.

```
ros2 launch f1tenth_stack bringup_launch.py
```

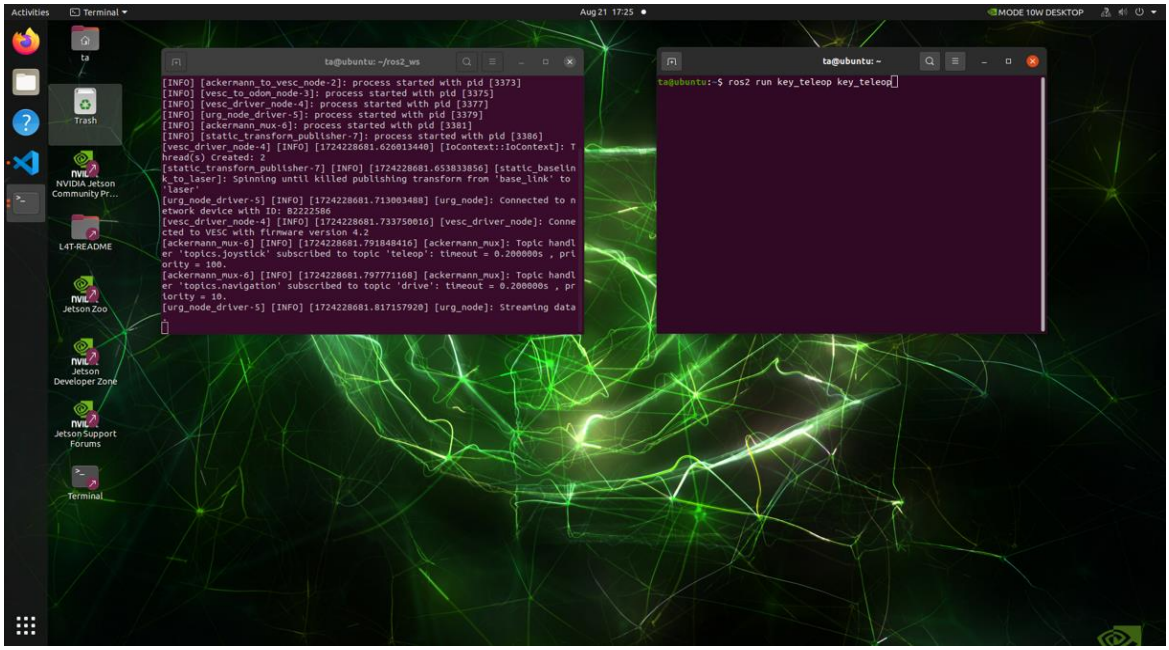
Caution : Before run key_teleop, you must configure the current settings in Motor Settings > General > Current of the VESC Tool exactly as shown in the screen below for safety.



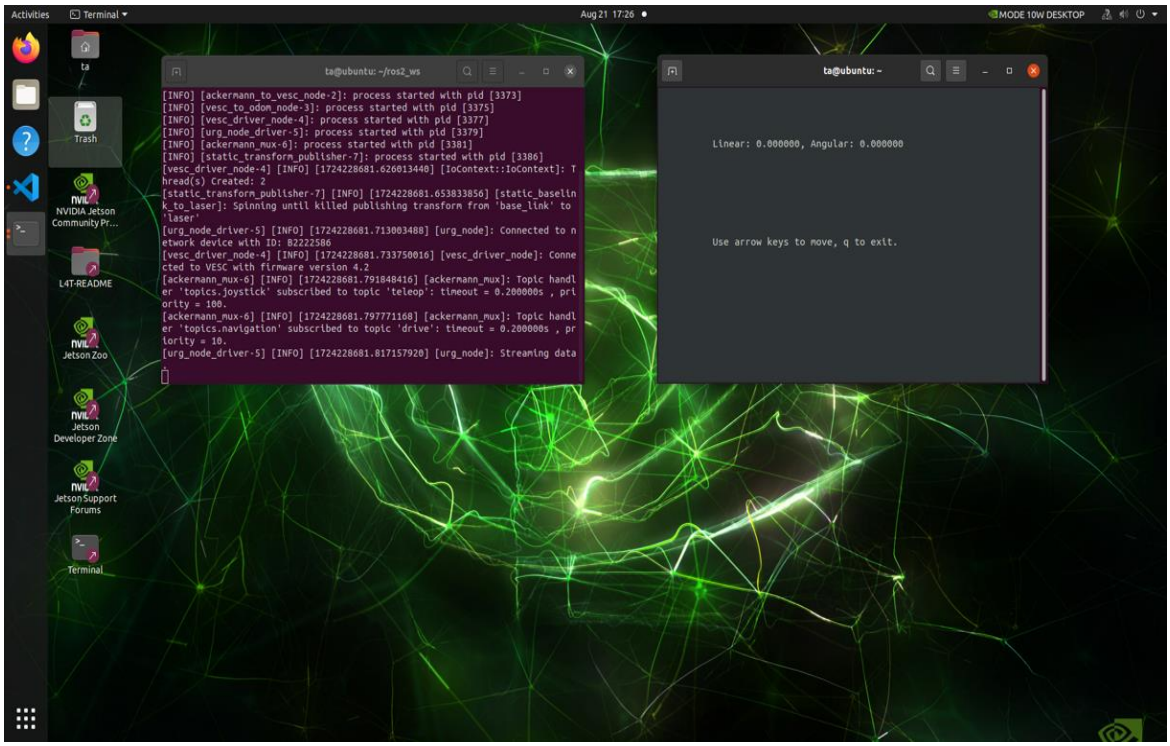
Caution : Make sure that you have set your ROS_DOMAIN_ID correctly before running the system.

Open the other terminal and run:

```
ros2 run key_teleop key_teleop
```



When you run the key_teleop node, you can see the black screen on the terminal.
Push button '**s**', and you can now control your RC car with the arrow buttons on the keyboard.



3. About RC car ROS 2 system code

In the subsequent section, we will begin by elucidating the interconnections between the ROS nodes through an analysis of the system's graph figure. Following this, we will provide a detailed explanation of the code underlying the key nodes.

3-1. Overview of the system

You can see the overview of node connections contained in the system as a graph.

```
rqt_graph
```

The message written between nodes represents the topic name that sends it to the connected node. Since your current agent_node code is incomplete, the graph displayed with the rqt_graph command will look like Figure 3.1. Once you have fully implemented the agent_node, you should see a graph like Figure 3.2.

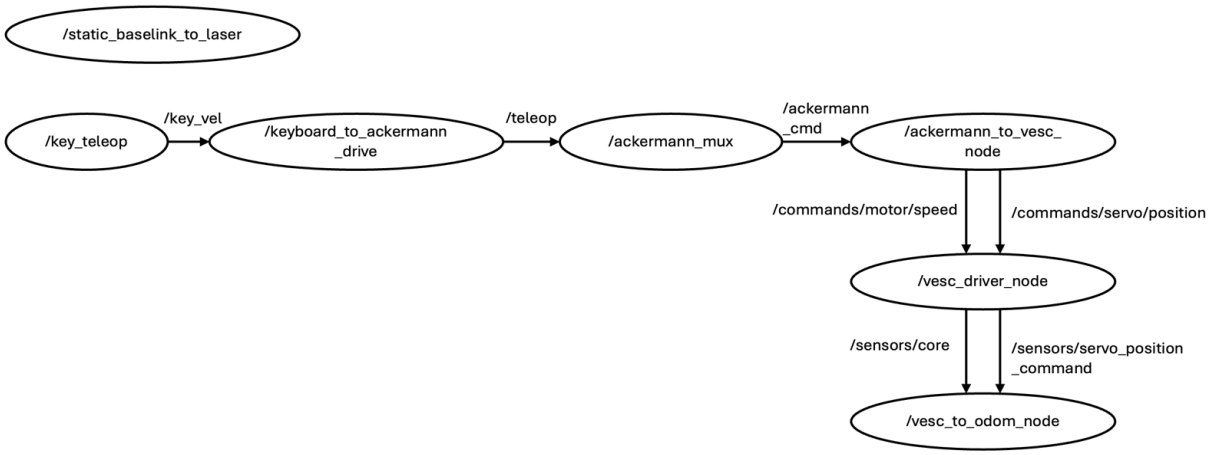


Figure 3.1. Initial rqt_graph

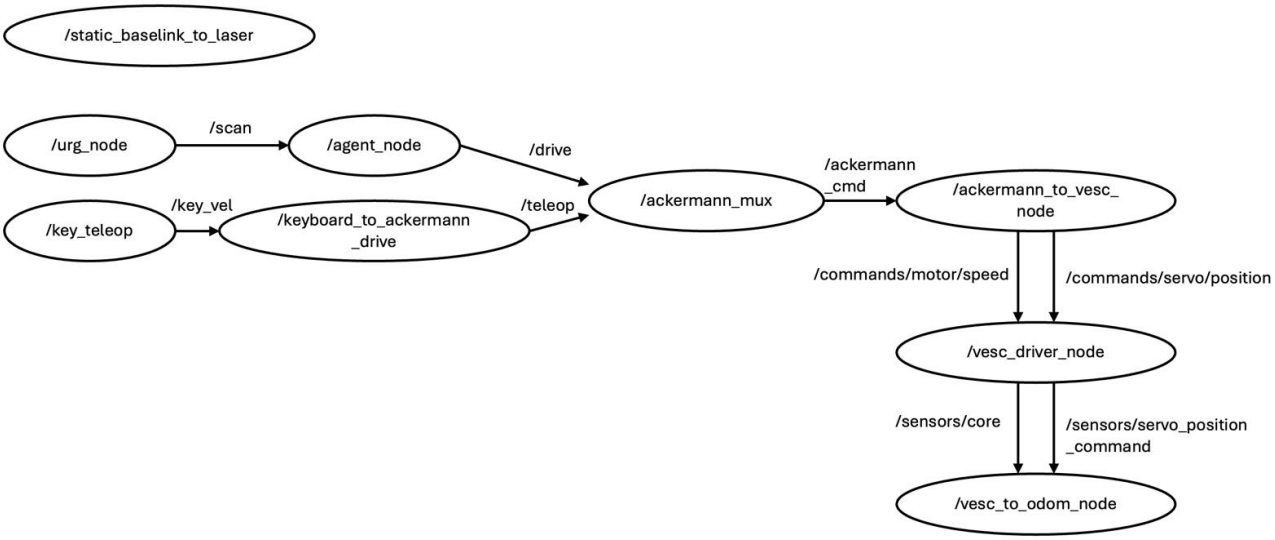


Figure 3.2. rqt_graph after writing agent_node

The agent_node is a node that uses the lidar input received by the autonomous agent to output a /drive message containing velocity and steering information. The /keyboard_to_ackermann_drive node is a node that takes keyboard input and outputs a /teleop message containing velocity and steering information. The ackerman_mux node prioritizes the /teleop message for keyboard control, choosing it over the /drive message for autonomous driving from agent_node.

To make sure that your Lidar has been connected correctly, run

```
ros2 topic echo /scan
```

You can see some list of outputs consist of LiDAR scan values.

3-2. Important nodes

```
ros2_ws
|— build
|— install
|— src
  |— f1tenth_system
    |— ackermann_mux
    |— f1tenth_stack
      |— f1tenth_stack
        |— agent_node.py
      |— launch
        |— bringup_launch.py
    |— scripts
    |— teleop_tools
      |— key_teleop
        |— key_teleop
          |— key_teleop.py
    |— vesc
```